2026 IEEE 6th International Conference on Smart Information Systems and Technologies (SIST)
May 13-15, 2026
Astana, Kazakhstan

Paper ID-83

Section: "Computer science, IEEE Computer Society (CS), IEEE Transactions on Computers. Performance, fault tolerance, reliability, security, and testability"

Customizable methodology for structured analysis of software within isolated computational environments to secure critical IT infrastructures

Authors' names: Noyan Tendikov, Rostyslav Lisnevskyi, Leila Rzayeva

University/company: Astana IT University
City, Country: Astana, Kazakhstan

Introduction

- Modern cyber conflicts within an intensified arms race between multiple different actors, such as government agencies, armies, corporations, and private groups.
- Under these circumstances, ensuring security is only a continuous process, not an absolute guarantee. It is import to to assess associated risks for critical IT infrastructure.
- Research goal and objectives – to construct theoretical basis of security measures for the IT infrastructure, emphasizing the process of collecting and accounting of its internal resources or components for reliability and early insider threat prevention.

Complexity management in IT infrastructures

- The computing ecosystem and its service models have evolved gradually becoming increasingly commercialized to meet customer demands and scaling issues.
- As IT infrastructure has expanded with its complexity. For example, codification established the paradigm of Infrastructure-as-Code (IaC) to provide maintenance by means of self-documenting code, as shown in Figure 1.

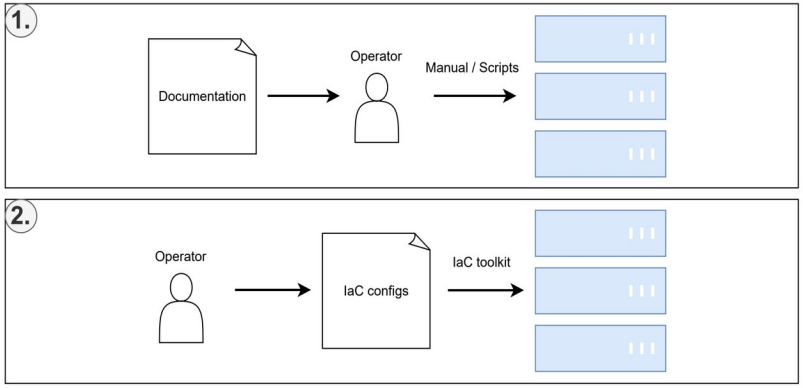


Figure 1: Processes of applying changes to servers: (1) Manually. (2) Via IaC pipeline as a self-documenting system.

Resource management and accounting

OWASP identified 10 major risks for IT infrastructure in which 5 of them are directly related to IT resources accounting [1]. Usually, solution-specific logic represents about 10% of code while the remainder is third-party dependencies [2].

- Software composition analysis (SCA) acts as structured approach for collecting and accounting software components.
- As shown in Figure 2, a script in the Python programming language is only a internal dependency on the surface, where everything else is third-party components.

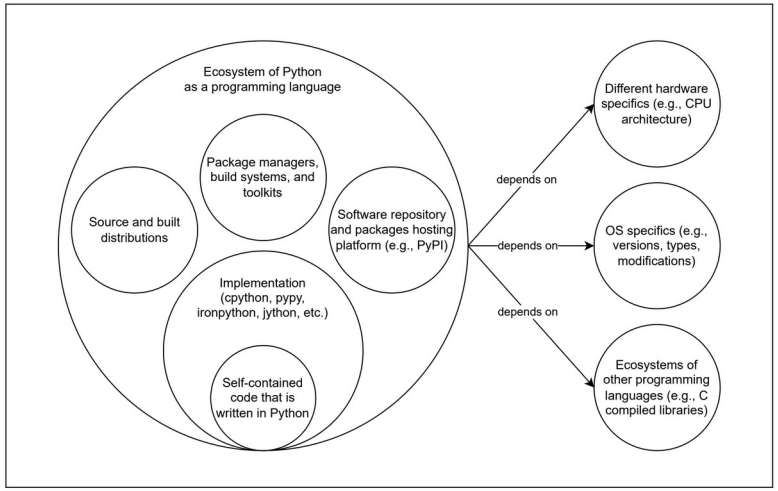


Figure 2: Ecosystem of the Python programming language from the perspective of dependencies.

Supply chain attacks

- The modern economy is service-oriented and highly interconnected in the global supply chain, where everyone relies on each other to mitigate issues of resource constraints by integrating external dependencies to solve common problems, as represented in Figure 3.
- Any risks and failures in a single link can cause a cascading effect across the entire chain, exposing the ability to implicitly or explicitly influence on any IT infrastructure by leveraging direct and indirect (i.e., transitive) dependencies, as demonstrated in Figure 4.

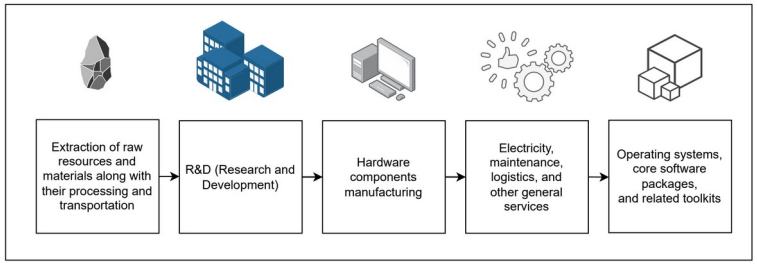


Figure 3: Simplified example of the global supply chain in the IT context.

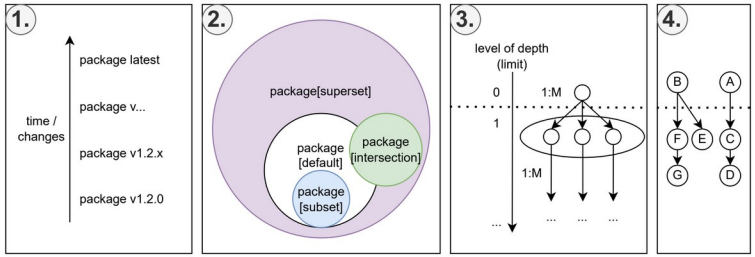


Figure 4: Dependency resolution is a variation on the topic of the constraint satisfaction problem where the objective is to satisfy a predefined set of conditions and rules (version requirements or other ecosystem-specific rules).

Compromise and trust

- Popular or operation-critical software products have a strong influence on the whole landscape, affecting on dependent infrastructure and global contracts of security, as shown in Figure 5.
- Therefore, trust should be viewed as multiple models:
  - Static trust is a relationship model that makes us vulnerable due to reliance on a third party without any doubt due to assumptions and distractions, considering them trustworthy.
  - Dynamic trust is a dynamic probability of being trusted by someone else and vice versa, where any mistake from both parties leads to its loss by some amount [3].
  - Zero trust requires continuous verification evaluation of all actions and actors instead of granting full access within a protected perimeter, assuming operating in a hostile environment [4].

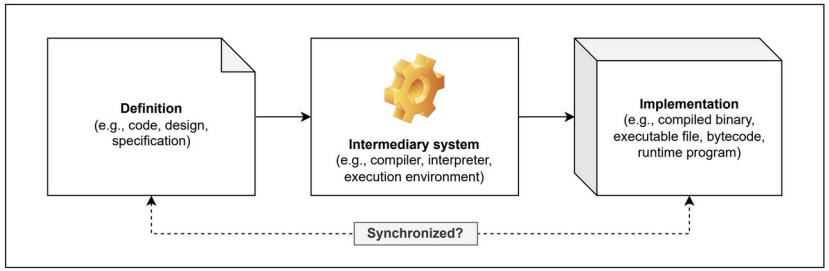


Figure 5: An intermediary system between design and implementation.

Theoretical methodology

- As illustrated in Figure 6, we propose a paradigm shift by redirecting the purpose of existing solutions of malware analysis toward regular developers to gain in-depth insights of potential threats. This form of software examination is attended within isolated execution environments as preventive mechanism and continuous security lifecycle for any component in IT infrastructure.

- It is only a theoretical concept without strict details on technical aspects to be technology-agnostic and portable to other ecosystems.

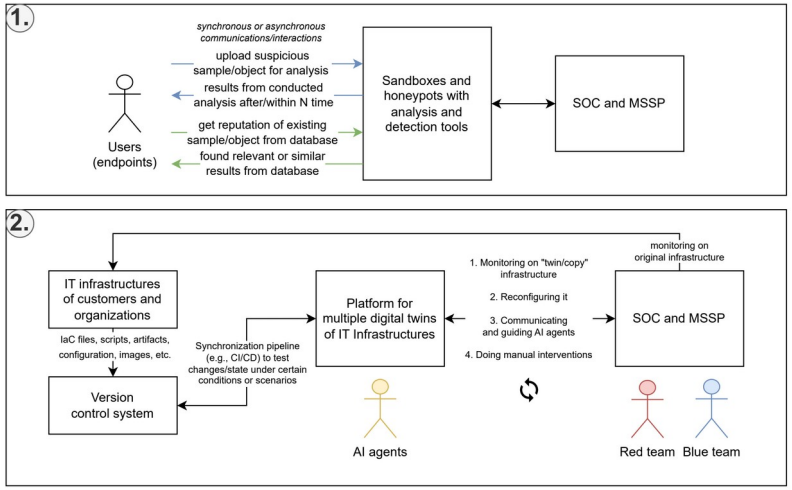
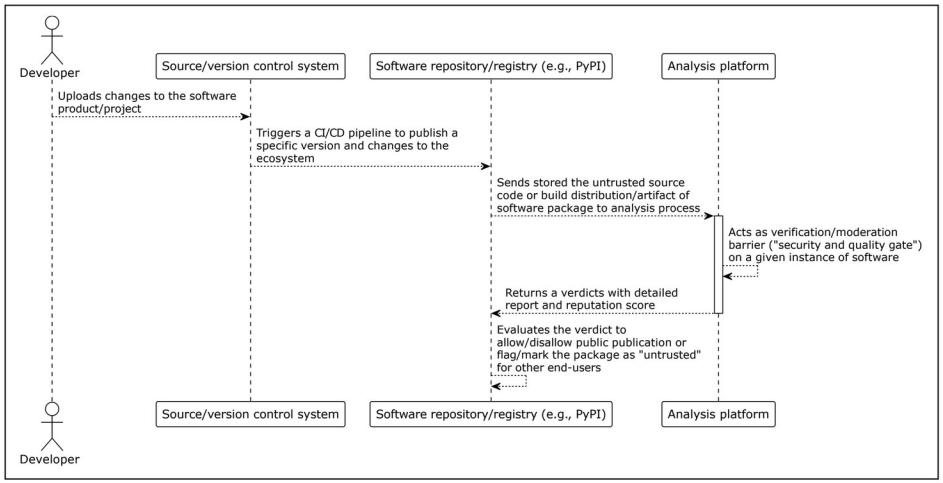


Figure 6: Transition to cycle of continuous security (SOC - security operations center; MSSP - managed security service provider): (1) Passive analysis with sandboxes and honeypots. (2) Reactive analysis with digital twins of IT infrastructures with AI agents.

Results by applying conceptual methodology to PyPI

- Components within the analysis engine:
  - developed for specific use cases related to software packages, container images, computer networks, and other parts of the IT infrastructure, as demonstrated for in Figure 7 and 8.
  - can be implemented differently but must intercommunicate via JSON-RPC 2.0, and interfaces describe their actions and roles operating around the abstraction of ”task”
  - default components interfaces: Orchestrator, Instancer, Collector, Interactor, Parser, and Analyzer.



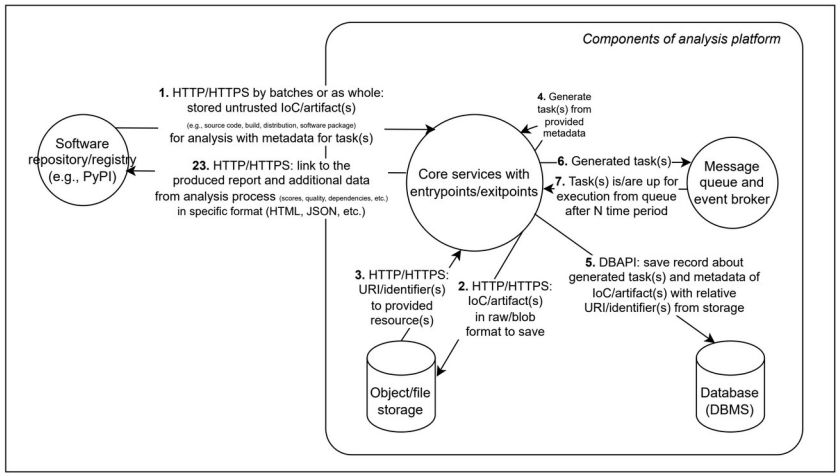


Figure 7 and 8: Secure software pipeline by integrating the proposed analysis platform to PyPI (Python packages distribution platform).

Results with limitations and conclusion

- Ensuring concurrency of their multiple instances via asynchronous inter-process communications and to prevent bottlenecks.
- The integration of AI agents as “Interactors” remains unreliable due to vulnerabilities such as cross-prompt injection without proper guardrails.
- Sandboxed environments represent only a limited model that diverges from real production systems.
- Continuous updates to ensure the analysis environment remains secure.
- Supply chain management in critical IT infrastructure remains highly relevant. There should be a natural shift toward preventive measures during the initial production and distribution of software packages.
- Further work is aimed at the complete practical implementation of the system and a detailed analysis of software components to actually verify the suitability of the conceptual framework using real data and gathered statistics

References

- Source files are provided in:
  - https://github.com/NoyanTM/aitu-sist-2026_research-paper
  - Write to me as @noyantendikov in Telegram and to Github issues, if you are interested in contributions and participation

- 1. OWASP/Www-Project-Top-10-Infrastructure-Security-Risks. OWASP, Oct. 22, 2025. url: https://github.com/OWASP/www-project-top-10-infrastructure-security-risks (visited on 11/05/2025).
- 2. European Cyber Security Organisation (ECSO). Technical Paper: Software Supply Chain Security. Tech. rep. WG6. European Cyber Security Organisation, Sept. 2024. url: https://ecs-org.eu/ecso-uploads/2024/09/ECSO-WG6-Technical-Paper-%E2%80%94-Software-Supply-Chain-Security.pdf.
- 3. Abe Singer and Matt Bishop. “Trust-Based Security; Or, Trust Considered Harmful”. In: New Security Paradigms Workshop 2020. NSPW ’20: New Security Paradigms Workshop 2020. Online USA: ACM, Oct. 26, 2020, pp. 76–89. isbn: 978-1-4503-8995-2. doi: 10.1145/3442167.3442179. url: https://dl.acm.org/doi/10.1145/3442167.3442179 (visited on 12/01/2025).
- 4. Scott Rose et al. Zero Trust Architecture. National Institute of Standards and Technology, Aug. 11, 2020. doi: 10.6028/NIST.SP.800-207. url: https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-207.pdf (visited on 11/10/2025).